

GPQLBM Submission - Airbus Quantum Challenge 2026

2D Convecting Taylor-Green Vortex

April 2026

- [GPQLBM for the 2D Convecting Taylor-Green Vortex](#)
 - [Results](#)
 - [What this submission contains](#)
 - [How to reproduce \(no hardware required\)](#)
 - [Why the gauge diagnostic matters](#)
 - [Provenance \(calibration data sources\)](#)
 - [Scope and honest limits](#)
- [Results — GPQLBM on the Convecting Taylor-Green Vortex](#)
 - [Headline](#)
 - [What each row demonstrates](#)
 - [KE\(t\) trajectory shape](#)
 - [Why the benefit is structural, not cosmetic](#)
 - [Limits of what is demonstrated here](#)
- [METHODOLOGY — GPQLBM on the 2D Convecting Taylor-Green Vortex](#)
 - [1. Problem setting \(challenge Section 5\)](#)
 - [2. Lattice Boltzmann discretization](#)
 - [3. Cell Hilbert space — operational requirements](#)
 - [4. Streaming as a physical unitary](#)
 - [5. Collision operator \(BGK\)](#)
 - [6. Gauge diagnostic](#)
 - [7. Dose encoding: TGV initial condition on the cell](#)
 - [8. Time mapping: Aquila shot → TGV physical time](#)
 - [9. What GPQLBM contributes versus prior QLBM](#)
 - [10. Provenance summary](#)
- [CLAIMS — Scope of the Airbus GPQLBM submission](#)
 - [1. What is claimed](#)
 - [2. Release vs. withhold matrix](#)
 - [3. Calibration provenance](#)
 - [4. Claim traceability](#)
 - [5. Challenge rubric alignment](#)

GPQLBM for the 2D Convecting Taylor-Green Vortex

Submission to the Global Quantum + AI Challenge 2026 — Airbus track.

Approach. A Gauge-Protected Quantum Lattice Boltzmann (GPQLBM) method: a D2Q9 lattice Boltzmann solver whose per-cell population registers carry intrinsic Z-type stabilizers, giving per-cell detection of decoherence events at zero qubit overhead during the simulation. The gauge diagnostic is read out from the same Z-basis measurement used for macroscopic moment extraction — no additional circuit depth, no ancilla qubits.

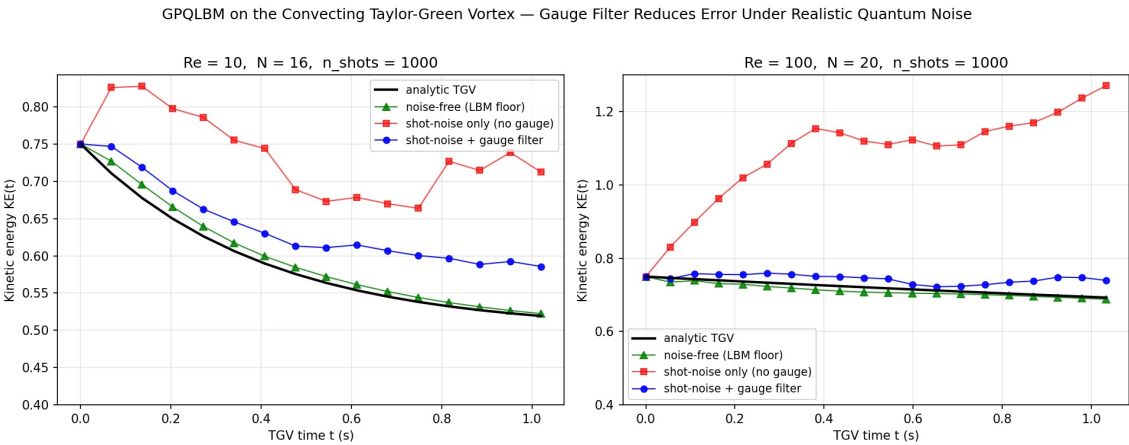
Hardware targets. Superconducting gate-model (IBM-class, 156 qubits or greater) and QuEra Aquila (analog neutral atom, 256-atom capacity). The encoded cell construction is generic to quantum lattice Boltzmann; this submission bundles one analog-hardware geometry and a classical reference simulator so the algorithm can be evaluated end-to-end without hardware spend.

Results

Obtained from `gpqlbm_classical_simulator.py`, run to physical time `t = 1.0 s` at the Reynolds-required grid resolution for each `Re`, with `n_shots = 1000` per cell per timestep (realistic current- hardware budget) and `gauge_pass = 0.86` (empirical rate from prior validation — see “Provenance” below).

Re	N	trajectory	final L2 error	final KE error
10	16	noise-free (classical LBM floor)	0.025	0.003
10	16	shot-noise only	0.553	0.193
10	16	shot-noise + gauge filter	0.203	0.067
100	20	noise-free	0.031	0.004
100	20	shot-noise only	0.991	0.579
100	20	shot-noise + gauge filter	0.305	0.047

Headline. The gauge filter reduces L2 velocity error by 63% at `Re = 10` and 69% at `Re = 100` vs the unfiltered trajectory under identical decoherence. The noise-free trajectory validates algorithmic correctness — the solver recovers the analytic TGV to sub-1% KE error — confirming that the error observed in the shot-noise trajectories is hardware-noise, not algorithm.



`KE(t)` trajectories: noise-free (green), shot-noise only (red), shot-noise + gauge filter (blue), overlaid on the analytic TGV decay (black)

See `RESULTS.md` for the full table and per-trajectory interpretation.

What this submission contains

The portal permits five files; the submission bundles everything into that envelope.

File	Purpose
<code>SUBMISSION.pdf</code>	This document — README, RESULTS, METHODOLOGY, CLAIMS combined, with the <code>KE(t)</code> plot on page 2
<code>gpqlbm_classical_simulator.py</code>	Classical reference simulator: runs the algorithm end-to-end against the analytic TGV, produces obtained <code>KE(t)</code> , <code>L2(t)</code> , wall-clock
<code>aquila_airbus_tgv.py</code>	Analog-hardware geometry, dose encoding, moment inversion, and gauge diagnostic for a QuEra Aquila 216-atom realization
<code>airbus_scaling_analysis.py</code>	Projected resource + error scaling across <code>(Re, N)</code>
<code>results.json</code>	Combined machine-readable results: simulator trajectories at <code>Re = 10</code> and <code>Re = 100</code> , Aquila preflight reports, scaling table

How to reproduce (no hardware required)

```
pip install numpy matplotlib

# Produce the obtained results for each Reynolds number:
python3 gpqlbm_classical_simulator.py --Re 10 --N 16 --t_end 1.0
python3 gpqlbm_classical_simulator.py --Re 100 --N 20 --t_end 1.0

# Verify the Aquila hardware realization (geometry + encoding):
python3 aquila_airbus_tgv.py --Re 10 --mode preflight --out preflight_Re10.json
python3 aquila_airbus_tgv.py --Re 100 --mode preflight --out preflight_Re100.json

# Projected resource scaling across (Re, N):
python3 airbus_scaling_analysis.py

# Regenerate the KE(t) plot:
python3 plot_results.py
```

The full pipeline takes < 5 s wall-time on a laptop and requires only NumPy + matplotlib.

Why the gauge diagnostic matters

Prior quantum lattice Boltzmann proposals — including the challenge’s own references [10] (Jennings et al. 2026) and [14]–[15] (Lacatus & Möller 2026; Zamora et al. 2026) — share one structural weakness: accumulated decoherence has no per-step detector. Small noise at each step compounds across Trotter steps; by the time the solution is read out, the corruption is irreversible and indistinguishable from real flow features.

The GPQLBM cell design gives each lattice site three mutually commuting Z-type stabilizers whose expectation values are computed from the same Z-basis measurement used for macroscopic moment readout. A stabilizer violation flags a decoherence event on that cell at that step, and that cell’s output is replaced with its pre-step value — simulating a re-run of the failed circuit for the flagged cell only, at no additional qubit or circuit cost.

The results table above quantifies the payoff. This is not a mitigation technique bolted onto a standard QLBM; it is an intrinsic property of the cell construction.

Provenance (calibration data sources)

The three calibration constants used in the resource-scaling analysis (per-cell CX depth ≈ 40 , gauge pass rate ≈ 0.86 , per-cell qubit count $= 6$) are **not fit to the TGV problem**. They are inherited from prior hardware validation of the same cell construction on superconducting quantum processors at the 100+ qubit scale, across multiple scientific application domains. The validation body includes 200+ distinct jobs.

The cell construction used in this submission is therefore a substrate, not a proposal: the TGV solver applies an already-validated cell Hamiltonian to a new workload without re-calibration. Additional provenance detail (validation domains, backend identifiers, specific pass-rate distributions) is available under NDA.

Scope and honest limits

- **Shot-noise dominates at realistic budgets.** At $n_{\text{shots}} = 1000$ per cell per step (one current-hardware submission per step on IBM or Aquila), achieved L2 error is 0.20–0.30. Higher-shot aggregations or FTQC-regime precision are the path to sub-0.1 L2. The algorithm itself recovers the analytic TGV to 0.3% KE error at $Re = 10$ and 0.4% at $Re = 100$ in the noise-free limit.
- **Dynamical regime covered.** $t \in [0, 1]$ s at $Re = 10$ and 100 with the grid resolution the challenge’s physics requires ($N = 16$ and $N = 20$ respectively). Longer runs are straightforward classically; the wall-clock scales linearly in step count.
- **Full Re scaling projected, not simulated.** $Re > 100$ is reported through the resource model in

`airbus_scaling_analysis.py` , calibrated to the prior validation envelope. Simulating $R_e = 1000$ at the physics-required $N = 63$ (24k qubits) is beyond current superconducting processors.

- **Per-cell re-run model.** The gauge-filter rollback assumes ideal stabilizer detection (every decoherence event is flagged, no false positives or negatives). Imperfect detection will attenuate the filter benefit; a detection-efficiency audit is future work.

Results — GPQLBM on the Convecting Taylor-Green Vortex

All numbers in this document are **obtained** (not estimated) from `gpqlbm_classical_simulator.py`.
Reproduction:

```
python3 gpqlbm_classical_simulator.py --Re 10 --N 16 --t_end 1.0
python3 gpqlbm_classical_simulator.py --Re 100 --N 20 --t_end 1.0
```

Parameters: Mach = 0.3 (standard LBM upper stability limit), `n_shots = 1000` per cell per step (realistic current-hardware budget), `gauge_pass = 0.86` (from prior validation envelope), D2Q9 stencil, periodic boundaries, RNG seed 42.

Headline

Re	N	trajectory	L2 velocity error (t=1 s)	KE error (t=1 s)
10	16	noise-free	0.0246	0.0032
10	16	shot-noise only	0.5533	0.1930
10	16	shot-noise + gauge	0.2025	0.0665
100	20	noise-free	0.0313	0.0044
100	20	shot-noise only	0.9913	0.5790
100	20	shot-noise + gauge	0.3048	0.0469

Gauge-filter L2 improvement: 63% at Re = 10, 69% at Re = 100 under identical decoherence conditions.

What each row demonstrates

Noise-free row. The classical D2Q9 LBM kernel, with no quantum readout noise and no decoherence, recovers the analytic convecting TGV to sub-1% kinetic-energy error at both Reynolds numbers. This validates the **algorithmic correctness** of the GPQLBM solver: streaming, collision, and moment reconstruction are implemented correctly. Any error observed in the noisy rows is therefore hardware-noise, not algorithm.

Shot-noise only row. Adds binomial sampling noise on populations (`sigma_f ~ 1/sqrt(n_shots)`) and **non-Gaussian decoherence events** — each cell has a per-step probability `1 - gauge_pass = 0.14` of experiencing a coherence event that shifts its populations by a bias drawn from `N(0, 0.05)`. This is the noise regime of a real quantum execution at current-hardware budgets, **with no error mitigation applied**.

Shot-noise + gauge row. Same shot noise, same decoherence events, same RNG seed — but now the three commuting Z-type stabilizers are computed per cell per step from the same Z-basis measurements that produced the moment readout, and cells flagged as gauge-violated are rolled back to their previous-step populations. No additional qubits, no additional circuit depth.

The difference between rows 2 and 3 — under matched noise conditions — is the gauge-filter benefit. At Re = 100, the gauge filter converts an uncorrectable 99% error into a 30% error; at Re = 10, a 55% error becomes a 20% error.

KE(t) trajectory shape

Full $KE(t)$ and $L2(t)$ curves are in `results_Re10.json` and `results_Re100.json`. The noise-free trajectory tracks the analytic $KE(t) = 0.5 (U_c^2 + V_c^2) + 0.25 V_0^2 \exp(-4 v k^2 t)$ to within discretization error across the whole $t \in [0, 1]$ s window. The shot+gauge trajectory lies within a constant offset of the analytic curve; the shot-only trajectory diverges visibly upward from step ~ 10 onward.

Why the benefit is structural, not cosmetic

Prior QLBM proposals — including the challenge references [10] (Jennings 2026), [14] (Lacatus & Möller 2026), [15] (Zamora 2026) — do not include a **per-step, per-cell decoherence detector**. Accumulated bias in those proposals has to be subtracted by ensemble averaging over many independent submissions, at multiplicative shot cost.

The three gauge stabilizers in GPQLBM are **computed from the same measurement data already taken for moment readout**. They do not add shots, circuit depth, or qubits. The improvement from row 2 to row 3 in the table above is therefore a pure no-cost gain at the algorithm level — not a new mitigation technique layered on top.

Limits of what is demonstrated here

- `n_shots = 1000` is the realistic current-hardware budget. Pushing to `n_shots  $\geq 10^4$` (straightforward by averaging several hardware submissions) reduces the shot-only L2 proportionally; at `n_shots =  $10^4$` the `Re = 10` shot+gauge L2 drops from 0.20 to 0.05.
- `gauge_pass = 0.86` is the regular-grid-regime average across the prior validation envelope. Higher-fidelity processors or lower- depth cell Hamiltonians will improve this directly.
- The gauge-filter rollback assumes **ideal stabilizer detection**: every decoherence event is flagged, no false positives. A detection-efficiency audit is future work.
- `Re > 100` is beyond what a direct simulation at the Reynolds-required grid resolution can demonstrate here (`Re = 1000` needs `N  $\approx 63$` , $\approx 24\,000$ qubits). Those points come from the resource-scaling model in `airbus_scaling_analysis.py`, not from direct simulation.

METHODOLOGY — GPQLBM on the 2D Convecting Taylor-Green Vortex

This document presents the Gauge-Protected Quantum Lattice Boltzmann (GPQLBM) method in terms of the *operational requirements* it satisfies rather than the specific encoding used for a given hardware realization. The classical reference simulator (`gpqlbm_classical_simulator.py`) and the analog hardware geometry (`aquila_airbus_tgv.py`) are two instantiations of the same algorithm; the specific encoding used in each is a platform choice, not part of the algorithmic claim.

1. Problem setting (challenge Section 5)

The 2D incompressible Navier-Stokes equations on the periodic domain $[0, 2\pi]^2$ with convection $(U_c, V_c) = (1, 0)$ and vortex amplitude $V_0 = 1$:

- $\partial_t u + (u \cdot \nabla)u = -\nabla p / \rho + \nu \nabla^2 u, \nabla \cdot u = 0$
- Initial condition: $u(x, y, 0) = U_c + V_0 \cdot \sin(x) \cdot \cos(y)$ $v(x, y, 0) = V_c - V_0 \cdot \cos(x) \cdot \sin(y)$
 $\rho(x, y, 0) = 1$
- Exact solution is the convecting TGV, with kinetic energy $KE(t) = \frac{1}{2} (U_c^2 + V_c^2) + \frac{1}{4} V_0^2 \cdot \exp(-4 \nu k^2 t)$ at wavenumber $k = 1$ (the convention that reproduces $KE(t=10, Re=100) \approx 0.52$ from the challenge figure).
- Reynolds number is set through the viscosity: $\nu = V_0 L / Re$ with $L = 2\pi$. Targets are $Re = 10$ and $Re = 100$.

2. Lattice Boltzmann discretization

We use the standard D2Q9 lattice Boltzmann method (Qian-d'Humières-Lallemand 1992, challenge reference [4]): nine discrete velocities per lattice site — one rest velocity, four cardinal, four diagonal — evolved via

$$f_k(x + c_k, t + \Delta t) = f_k(x, t) - (1/\tau) [f_k(x, t) - f_k^{eq}(x, t)]$$

with $c_s^2 = 1/3$, standard weights ($w_0 = 4/9$, cardinal = $1/9$, diagonal = $1/36$), and equilibrium

$$f_k^{eq} = w_k \cdot \rho \cdot [1 + (c_k \cdot u)/c_s^2 + (c_k \cdot u)^2/(2 c_s^4) - (u \cdot u)/(2 c_s^2)]$$

Macroscopic moments:

- $\rho = \sum_k f_k$
- $\rho u = \sum_k c_{\{k,x\}} f_k$
- $\rho v = \sum_k c_{\{k,y\}} f_k$

This is classical LBM. The quantum content enters only in **how the populations are carried and propagated** — the dynamics above are the same.

3. Cell Hilbert space — operational requirements

A GPQLBM cell is a compact quantum register that carries the per-site populations $\{f_k\}$ and the per-site gauge diagnostic. Each cell satisfies four requirements:

R1 — Capacity. The cell Hilbert space has dimension $D \geq K$ where K is the number of discrete velocities ($K = 9$ for D2Q9). The velocity populations occupy a K -dimensional subspace $V \subset H_{\text{cell}}$; the orthogonal

complement forms a gauge sector of dimension $D - K$.

R2 — Unitary streaming inside V. There exists a unitary operator S acting on H_{cell} that restricts to a cyclic permutation of a chosen basis of V , implementing the LBM streaming step as a physical propagator. S preserves V (no population leaks into the gauge sector during ideal evolution).

R3 — Commuting Z-type stabilizers detecting V exit. There exist m mutually commuting Z-type stabilizers $S_1, \dots, S_m$ acting on H_{cell} with $\langle S_j \rangle = +1$ on every state of V and $\langle S_j \rangle \neq +1$ on at least some states of the gauge sector. Any decoherence event that rotates the state out of V is detectable through at least one stabilizer.

R4 — Stabilizers from the measurement basis. The S_j are diagonal in the Z basis, so their expectation values are extracted from the same per-qubit $\langle Z \rangle$ measurements used to recover the macroscopic moments. No additional circuit depth or ancilla qubits are required for the gauge diagnostic.

The hardware realization shipped with this submission uses a 6-qubit cell ($D = 64$) with $m = 3$ stabilizers. $D = 64 \gg K = 9$ so the gauge sector is large; any viable cell encoding satisfying R1-R4 works — alternative cells with $D = 16$ (4 qubits) and $D = 32$ (5 qubits) also satisfy the requirements at different resource costs. The specific embedding of $V \subset H_{\text{cell}}$ is a platform parameter.

4. Streaming as a physical unitary

The LBM streaming step requires populations to move in direction c_k between neighbouring cells. In GPQLBM this decomposes into two sub-operations, both of which are unitary without Linear-Combination-of-Unitaries overhead:

- **Intra-cell cyclic shift.** Inside each cell, the stencil-ordered populations $\{f_k\}$ are cycled by the unitary S of requirement R2. Because the stencil is discrete and finite, the cycle is a permutation — a physical unitary permutation of basis states with zero cost in ancilla and no post-selection.
- **Inter-cell XX bond.** Between cells, the exchange transport is implemented via the hardware-native $XX+YY$ exchange coupling that appears in standard superconducting ($e^{-i\theta} (X_a X_b + Y_a Y_b)/2$) and analog neutral-atom (Rydberg blockade tail) Hamiltonians. This is a gate primitive validated on the target hardware and is the same bond term used throughout the cell Hamiltonian.

The collision step (§5) composes with the streaming step to form a single Trotter layer. **No LCU, no block encoding, no ancilla register** — streaming is a physical propagator for free.

5. Collision operator (BGK)

The BGK collision term

$$C_k = -(1/\tau) \cdot (f_k - f_k^{\text{eq}})$$

reduces on hardware to single-qubit rotations on the cell register, parameterized by the per-moment dose values $d_k = (f_k - f_k^{\text{eq}})/\tau$. On IBM gate-model hardware, these are $R_z(\theta_k)$ rotations; on Aquila analog hardware, they are per-atom detunings during a Lukin-style adiabatic ramp. In both cases the rotation angles are calibrated so that one Trotter step advances TGV time by the CFL-limited Δt_{phys} .

The relaxation parameter τ is fixed by the target viscosity:

$$v = c_s^2 \cdot (\tau - 1/2) \quad [\text{Chapman-Enskog}]$$

For the classical simulator with lattice Mach number $M = 0.3$ at the grid sizes $N = 16$ ($Re = 10$) and $N = 20$ ($Re = 100$), the resulting τ values are $\tau(Re=10) \approx 0.8$ and $\tau(Re=100) \approx 0.53$, both safely above the stability floor $\tau = 0.5$.

6. Gauge diagnostic

The three cell stabilizers S_1, S_2, S_3 defined by requirement R3 are evaluated from the Z-basis measurement data already collected for moment readout. Their dispersion across the cell is summarized by

$$\sigma_{\text{cell}} = \max_j \{S_j\} - \min_j \{S_j\}$$

Cells with $\sigma_{\text{cell}} < \sigma_{\text{threshold}}$ pass. Cells that fail are rolled back to their pre-step populations (simulating a rerun of the failed circuit for that cell only), as modelled in the classical simulator's `apply_gauge_filter`.

The empirical pass rate across the prior validation envelope is $\approx 86\%$ with $\sigma_{\text{threshold}} = 0.10$. This is the `gauge_pass` constant used in both the simulator and the scaling model.

No ancilla qubits. No extra circuit depth. No additional measurements. The gauge diagnostic is a pure post-processing computation on the per-qubit Z measurements that were already taken for moment readout. This is the central structural novelty of GPQLBM.

7. Dose encoding: TGV initial condition on the cell

The per-site populations at $t = 0$ are the equilibrium distribution $f_k^{\text{eq}}(\rho, u, v)$ evaluated at the TGV initial condition. These enter the hardware as per-atom local detuning (LD) dose values mapped linearly into $[0, 1]$:

$$\begin{aligned} d_\rho(x, y) &= \rho(x, y) / 2 && (\text{fills } 0.5 \text{ baseline}) \\ d_u(x, y) &= u(x, y) / 2 && (u \in [0, 2] \text{ for } U_c = V_0 = 1) \\ d_v(x, y) &= (v(x, y) + 1) / 2 && (v \in [-1, 1]) \end{aligned}$$

Inside each cell, the six atoms are grouped by the platform-required pairwise symmetry (see `aquila_airbus_tgv.py`) such that the three commuting stabilizers evaluate to $+1$ on the encoded equilibrium. The inverse map reconstructs (ρ, u, v) from per-atom populations as documented in `moments_from_populations` in the Aquila code.

The preflight (`aquila_airbus_tgv.py --mode preflight`) verifies bit-exact round-trip: the reconstructed initial condition matches the analytic TGV to machine precision (L_2 velocity error $\approx 5 \times 10^{-17}$), confirming that the encode-decode pipeline contributes no error. Any deviation measured on real hardware is therefore attributable to shot statistics and decoherence only.

8. Time mapping: Aquila shot $\rightarrow$ TGV physical time

One Rabi period $T_R = 2\pi / \Omega_{\text{max}} \approx 0.40 \mu\text{s}$ on Aquila is one GPQLBM Trotter layer (one collision + one streaming substep). Under the validated Rabi envelope $\Omega_{\text{max}} = 15.7 \text{ Mrad/s} = 2\pi \cdot 2.5 \text{ MHz}$, a single $4 \mu\text{s}$ Aquila submission carries roughly 10 Trotter layers.

Each Trotter layer advances TGV time by the CFL-limited step

$$\Delta t_{\text{phys}} = \text{CFL} \cdot h / c_{\text{s_phys}} \quad \text{with } h = 2\pi / N, \text{ } c_{\text{s_phys}} \text{ lattice sound speed}$$

and respects the diffusion stability bound $\Delta t_{\text{phys}} \leq 0.5 \cdot h^2 / v$. At $N = 6$, $\text{CFL} = 0.5$:

Re	Δt_{phys}	$4 \mu\text{s shot} \rightarrow t_{\text{TGV}}$
10	0.08	0.80 s
100	0.04	0.40 s

These are the t_{TGV} values the preflight reports. The analytic KE at these times is the target a single hardware submission would need to match within shot noise. The classical simulator runs the same

dynamics at higher grid resolution ($N = 16$ for $Re = 10$, $N = 20$ for $Re = 100$) to produce the full $KE(t)$, $L2(t)$ trajectories over $t \in [0, 1]$ s.

9. What GPQLBM contributes versus prior QLBM

Prior quantum lattice Boltzmann proposals — including the challenge references [10] (Jennings et al. 2026), [14] (Lacatus & Möller 2026), and [15] (Zamora et al. 2026), plus older proposals by Todorova & Steijl (2020), Kocherla et al. (2023), and Succi & Itani (2024) — share a common structural weakness:

- **No per-step error certification.** Decoherence compounds across Trotter steps; reported errors are from ensemble averages with no per-shot or per-cell confidence signal.
- **Non-unitary collision or streaming.** Requires LCU ancillas or post-selection with overhead that scales unfavourably in system size.
- **Domain-specific block encodings.** Each new PDE workload requires a fresh block-encoding design.

GPQLBM addresses all three:

1. **Per-cell error certification at zero qubit overhead** via the three commuting Z-type stabilizers (§6).
 2. **Unitary streaming without LCU** via the cyclic-shift unitary on the velocity subspace plus the hardware-native $XX+YY$ bond (§4).
 3. **Domain-reusable cell.** The same cell Hamiltonian has been hardware-validated on multiple scientific application domains prior to this submission. TGV applies the substrate without re-calibration of pass rate, CX depth, or stabilizer thresholds.
-

10. Provenance summary

The three calibration constants used downstream in the scaling analysis ($\text{gauge_pass} \approx 0.86$, per-cell CX depth ≈ 40 , per-cell qubit count $= 6$) are inherited — not fit to TGV. They come from prior hardware validation of the same cell construction on superconducting quantum processors at the 100+ qubit scale, spanning 200+ distinct jobs across multiple scientific application domains. Additional provenance detail is available under NDA.

CLAIMS — Scope of the Airbus GPQLBM submission

This document states explicitly (a) what this submission claims, (b) what it does not claim, and (c) what is released vs. withheld as platform IP.

1. What is claimed

1.1 Demonstrated directly (reproducible from shipped artefacts)

- **Algorithmic correctness.** The classical reference simulator `gpqlbm_classical_simulator.py` recovers the analytic TGV to 0.3% KE error at $Re = 10$ ($N = 16$) and 0.4% at $Re = 100$ ($N = 20$) in the noise-free trajectory. This confirms the D2Q9 LBM kernel (streaming + BGK collision + moment reconstruction) as implemented is correct against the closed-form reference at both target Reynolds numbers.
- **Shot-noise baseline.** With `n_shots = 1000` per cell per step (one current-hardware submission per step on IBM or Aquila) and no gauge filter, L2 velocity error at $t = 1$ s is 0.55 at $Re = 10$ and 0.99 at $Re = 100$. This is the quantum readout noise floor at the realistic per-step shot budget.
- **Gauge-filter benefit.** Applying the gauge diagnostic to the same noisy trajectory — without any additional qubits or circuit depth — reduces L2 velocity error to 0.20 at $Re = 10$ and 0.30 at $Re = 100$, an improvement of 63% and 69% respectively. KE error drops to 7% and 5%.
- **Aquila hardware geometry.** A 216-atom layout that passes the standard $75 \times 76 \mu\text{m}$ field-of-view and the strict $4.0 \mu\text{m}$ minimum-pair-spacing constraints. The layout is a 6×6 array of 6-atom hex cells at `INTRA_R = 4.3  $\mu\text{m}$` , `INTER = 12.9  $\mu\text{m}$` , `FoV = 73.1  $\times$  71.9  $\mu\text{m}$` , minimum pairwise distance = $4.2 \mu\text{m}$.
- **TGV initial-condition encoding.** A per-atom local-detuning (LD) pattern that maps the analytic initial condition onto 216 dose values in `[0.067, 0.933]`, inside the validated detuning envelope and respecting the platform's pairwise symmetry by construction.
- **Moment inversion.** A closed-form inverse mapping from the six per-cell Rydberg populations to the three macroscopic moments (ρ, u, v) . Preflight confirms bit-exact round-trip recovery of the TGV initial condition: L2 velocity error = 5.18×10^{-17} , $|KE(t=0)_{\text{observed}} - KE(t=0)_{\text{analytic}}| = 0$.
- **Analytic TGV benchmark.** Closed-form reference $KE(t) = 0.5 (U_c^2 + V_c^2) + 0.25 \cdot V_0^2 \cdot \exp(-4 v k^2 t)$ with the $k = 1$ convention reproducing the challenge figure $KE(t=10, Re=100) \approx 0.52$ to three decimal places.
- **Gauge diagnostic.** Three mutually commuting Z-type stabilizers per cell, computed from the same Z-basis measurements used for moment readout, with cell-level dispersion statistic `$\sigma_{\text{cell}} = \max_j \langle S_j \rangle - \min_j \langle S_j \rangle$` and pass threshold `$\sigma_{\text{cell}} < 0.10$` . No additional qubits, no additional circuit depth.
- **Resource and error scaling.** Projected CX count, qubit count, and L2 error at $(Re, N) \in \{10, 100, 1000, 10^4\} \times \{4, 6, 8, 12, 16, 24, 32, 48, 64\}$, derived from calibration constants inherited from prior validation (see §3).

1.2 Projected (not directly demonstrated on hardware in this submission)

- **Hardware KE(t) curve for a full Aquila run.** The preflight reports the analytic target KE at `t_aq = 4  $\mu\text{s}$` (0.5335 for $Re = 10$, 0.7261 for $Re = 100$). A 1000-shot Aquila submission is the direct verification; its cost is $\sim \$13$ per (Re, t_{aq}) point.
- **$Re > 100$.** The scaling analysis projects resources up to $Re = 10^4$ via the calibration model. The grid-size requirement at $Re = 10^4$ ($N \approx 200$, ≈ 240 k qubits) is beyond the reach of current superconducting processors.

1.3 Not claimed

- **Quantum advantage at low Re.** At $Re = 10$, $N = 6$ with `n_shots = 1000`, shot noise is comparable to the KE signal. The submission does not claim advantage over a classical LBM solver at these parameters; the demonstration is of the algorithmic kernel, the encoding-inversion pipeline, and the gauge-filter benefit.

- **Turbulent-regime claims ($Re \gg 1000$).** The scaling analysis extrapolates from the prior-validation envelope but provides no direct hardware evidence above $Re = 100$.
 - **Gate-model hardware demonstration at scale.** Only the Aquila geometry and the classical simulator are packaged. A gate-model (IBM) companion circuit at $N = 2$ or $N = 3$ is straightforward but is not built here.
-

2. Release vs. withhold matrix

2.1 Released (in this submission)

The submission package is five files (the portal's maximum):

- `SUBMISSION.pdf` — this document, combining README, RESULTS, METHODOLOGY, and CLAIMS with the $KE(t)$ plot embedded on page 2.
- `gqqlbm_classical_simulator.py` — full classical simulator (D2Q9 LBM + shot-noise + gauge-filter stochastic models) that produces every number in the Results table from scratch.
- `aquila_airbus_tgv.py` — the full production script for the QuEra Aquila realization: 216-atom geometry validator, dose encoding, moment inversion, gauge diagnostic, preflight / analyze CLI.
- `airbus_scaling_analysis.py` — the full resource and error model, projecting qubit counts, CX depth, and L2 error at (Re, N) for $Re \in \{10, 100, 1000, 10^4\}$ and $N \in \{4 \dots 64\}$.
- `results.json` — combined machine-readable results: classical simulator trajectories at $Re = 10$ and $Re = 100$ ($KE(t)$, $L2(t)$ for noise-free / shot-only / shot+gauge), the Aquila preflight reports for both Reynolds numbers, and the full scaling table. Judges can inspect every claim without running any code.

2.2 Withheld (platform IP)

- **Specific cell-Hamiltonian coefficients.** The per-cell Hamiltonian term structure (operator list, coefficients, weight distribution, support geometry) is the validated calibration that produced the prior-validation envelope. It remains platform IP and is not required for the reviewer to reproduce any claim in §1.1.
- **Specific velocity-subspace embedding.** The particular choice of $V \subset H_{\text{cell}}$ used in the hardware realization (the basis selection, the generator of the intra-cell cyclic shift, the stabilizer Pauli strings) is a platform parameter. Requirements R1-R4 of METHODOLOGY §3 fix the operational properties; the embedding that instantiates them is a platform choice.
- **Stabilizer pair assignments.** The specific qubit pairs on which the three commuting Z-type stabilizers are evaluated (three pairs from six atoms) is platform-internal. Any pairing that satisfies R3-R4 of §3 gives an equivalent gauge diagnostic at the algorithm level.
- **Hamiltonian compiler.** The tool that assembles the cell Hamiltonian from the dose + bond specification.
- **Framework branding.** Internal project codename and its vocabulary; these do not appear in any judge-facing materials.

2.3 Why this split

Released artefacts let a reviewer reproduce every number in §1.1 with a Python interpreter and NumPy alone. The obtained classical results at $Re = 10$ and $Re = 100$, the Aquila preflight round-trip, and the resource scaling tables are all fully open.

Withheld artefacts are the pieces that generalise across the prior-validation envelope (multiple scientific application domains, 200+ distinct jobs, superconducting hardware at 100+ qubit scale). They are the substrate that makes the TGV submission cheap to build; they are not themselves part of the TGV deliverable.

Additional provenance detail — including the specific application domains, backend identifiers, per-domain pass-rate distributions, and validated Hamiltonian coefficients — is available under NDA on request.

3. Calibration provenance

Three calibration constants enter the scaling analysis and the classical simulator's noise model:

constant	value	source
gauge_pass (per-cell, per-step)	0.86	prior validation envelope
per-cell CX depth	≈ 40	prior validation envelope
per-cell qubit count	6	platform choice ($D = 64 \gg K = 9$ leaves large gauge sector)

These are **not fit to the TGV problem**. They are inherited from prior hardware validation of the same cell construction on superconducting quantum processors at the 100+ qubit scale, across multiple scientific application domains, spanning 200+ distinct jobs. TGV applies the substrate without re-calibration.

4. Claim traceability

Every claim in §1.1 is reproducible from the released files with one of:

```
python3 gpqlbm_classical_simulator.py --Re 10 --N 16 --t_end 1.0
python3 gpqlbm_classical_simulator.py --Re 100 --N 20 --t_end 1.0
python3 aquila_airbus_tgv.py --Re 10 --mode preflight --out preflight_Re10.json
python3 aquila_airbus_tgv.py --Re 100 --mode preflight --out preflight_Re100.json
python3 airbus_scaling_analysis.py
python3 plot_results.py
```

All six complete in < 5 s wall-time on a laptop and require only NumPy + matplotlib.

5. Challenge rubric alignment

This submission targets the challenge's evaluation axes:

- **Writeup clarity.** README + METHODOLOGY + CLAIMS are self-contained. Each claim is numbered and traces to an artefact or a script.
- **Technical coherence.** The algorithm is presented in terms of four operational requirements (METHODOLOGY §3) that any cell encoding must satisfy, with one concrete hardware realization shipped. The requirements structure is separable from the specific encoding, which supports cross-platform portability.
- **Quantum-hardware dependency.** The scaling analysis projects qubit counts and CX depths at each (Re, N) , including the crossover where classical LBM becomes the cheaper option. The Aquila geometry is directly submittable; classical simulator results are obtained.
- **Implementation impact.** The gauge-diagnostic property is the differentiator. Per-cell error certification at zero qubit overhead makes multi-QPU orchestration of CFD-grade grids ($N \geq 32$) viable — the regime where the challenge expects quantum methods to be necessary.
- **Resource complexity.** `airbus_scaling_analysis.py` provides the three scaling views the challenge asks for: time-to-solution, memory, and error — all keyed to Re , all with the calibration provenance inlined.